



## JOBSHEET

### Case Study 2

#### 1. Objective

- Students are able to solve case study problems by applying the concept of Linear Data Structures.
- sorting and searching algorithms according to case studies.

#### 2. Practicum

**Processing Time: 200 minutes**

##### **Work Instructions:**

Each student **will code a program** based on a given case study. Ensure the program runs smoothly, implements all requested features, and is dynamic (using menus to select features).

##### **Case Study Questions:**

At lunch time, the Royal Delish restaurant has many customers coming at the same time. Cashiers have difficulty recording customer queues and food orders because the process is still done manually.

As a result:

- Queue numbers are often mixed up.
- Buyer data is not recorded neatly.
- Food orders are hard to recap.

To overcome this problem, a **Linked- based queue information system was created. List / Double Linked List .**

System flow:

- Buyers come and take a queue number.
- Buyer data is entered into the system.
- When called, the customer places a food order.
- Order data is saved.
- The system can display order reports and total restaurant revenue.

In this system there are two main classes , namely:

|                            |
|----------------------------|
| Buyer                      |
| Name : String              |
| MobileNumber : String      |
| Buyer( String b, String c) |

|                               |
|-------------------------------|
| Order                         |
| OrderCode : int               |
| OrderName : String            |
| price : int                   |
| Order(int a, String b, int d) |

The image above is an example of a class diagram of Buyers and Orders. Attributes and methods can be added.

Must-Have Features:

1. Add Queue

When you select the "Add Queue" menu, the system will add a queue number according to the existing queue order. Queue numbers can be generated automatically. The system will then ask for input, including the customer's name and phone number. The entered data will be saved in the linked queue. list.

Example:

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 1
Buyer Name : Zaki
No HP      : 09876543
Antrian berhasil ditambahkan dengan nomor: 4
=====
```

Note: Use data addition by creating objects using constructors to avoid time-consuming data input via keyboard .

2. Print Queue

This menu will display all the queues that are on linked list

Example :

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 2
=====
Daftar Antrian Pembeli
=====
```

| No Antrian | Nama  | No HP       |
|------------|-------|-------------|
| 1          | Ainra | 08224500000 |
| 2          | Danra | 08224511111 |
| 3          | Sanri | 08224522222 |
| 4          | Zaki  | 09876543    |

### 3. Clear Queue

Clear the queue must match the queue number. The clear queue menu indicates the customer has gone to the cashier to order food. Therefore, the customer's food order must be entered in this menu.

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 3
Order Code : 123
Order Name : Orange Juice
Price      : 25000
Ainra telah memesan Orange Juice
```

then the queue must be reduced. In the example below , there are already 3 customers who have ordered food so there are 1 people left in the queue



```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 2
=====
Daftar Antrian Pembeli
=====
```

| No Antrian | Nama  | No HP       |
|------------|-------|-------------|
| 2          | Danra | 08224511111 |
| 3          | Sanri | 08224522222 |
| 4          | Zaki  | 09876543    |

#### 4. Order report

This menu displays all incoming orders. Order data should be displayed sorted by order name.

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 4
=====
LAPORAN PESANAN (URUT NAMA PESANAN)
=====
```

| Kode Pesanan | Nama Pesanan | Harga |
|--------------|--------------|-------|
| 123          | Orange Juice | 25000 |



## Assessment Rubric

**Components: Code Assessment: 30% + CM Practical Test Assessment: 70%**

### Code Assessment Rubric

| No | Aspect Evaluation                   | Weight | Score 1                 | Score 2                  | Score 3                               |
|----|-------------------------------------|--------|-------------------------|--------------------------|---------------------------------------|
| 1  | Implementation Queue Data Structure | 20%    | Do not use linked lists | Using Single Linked List | Using Double Linked List with Correct |
| 2  | Implementation Order Data Structure | 20%    | Do not use linked lists | Using Single Linked List | Using Double Linked List with Correct |
| 3  | Added Features Queue                | 10%    | Not running             | Walk part                | Walk with Good                        |
| 4  | Print Features Queue                | 10%    | Not showing             | Showing some data        | Showing all data with Correct         |
| 5  | Queue & Order Input Features        | 15%    | Not running             | Walk part                | Walk in accordance question           |
| 6  | Report Order & Sorting              | 20%    | No sorting              | Sorting less appropriate | Sorting in progress Correct           |

### Additional Terms

Programs may not use Collections Frameworks such as:

- ArrayList
- Linked List
- Queue
- Stack
- Collections.sort ()
- Sorting is done manually.

The program must use objects and constructors .

Double Use Linked The list in both data structures will get maximum value in the data structure aspect.